第十三章 OpenGL视图

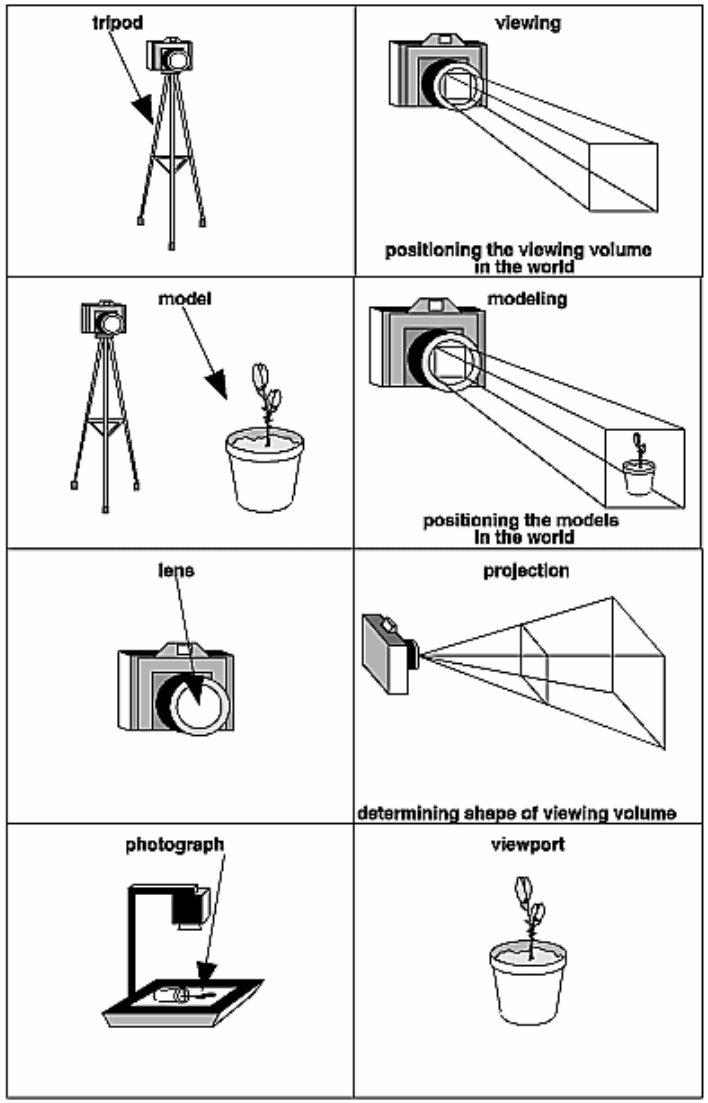
本章目的

- 根据不同的视点位置显示模型
- 控制模型在三维空间中的位置
- 剪裁视区外的几何模型
- 操作矩阵堆栈控制模型的视图变换和投影变换
- 结合多种变换来模拟复杂运动系统
- 操作逆转矩阵

内容简介

- 照相机类比法
- 视图变换和模型变换
- 投影变换
- 视区变换
- 变换的诊断
- 矩阵堆栈的操作
- 附加剪裁平面
- 组合变换举例
- 逆转矩阵

照相机类比法

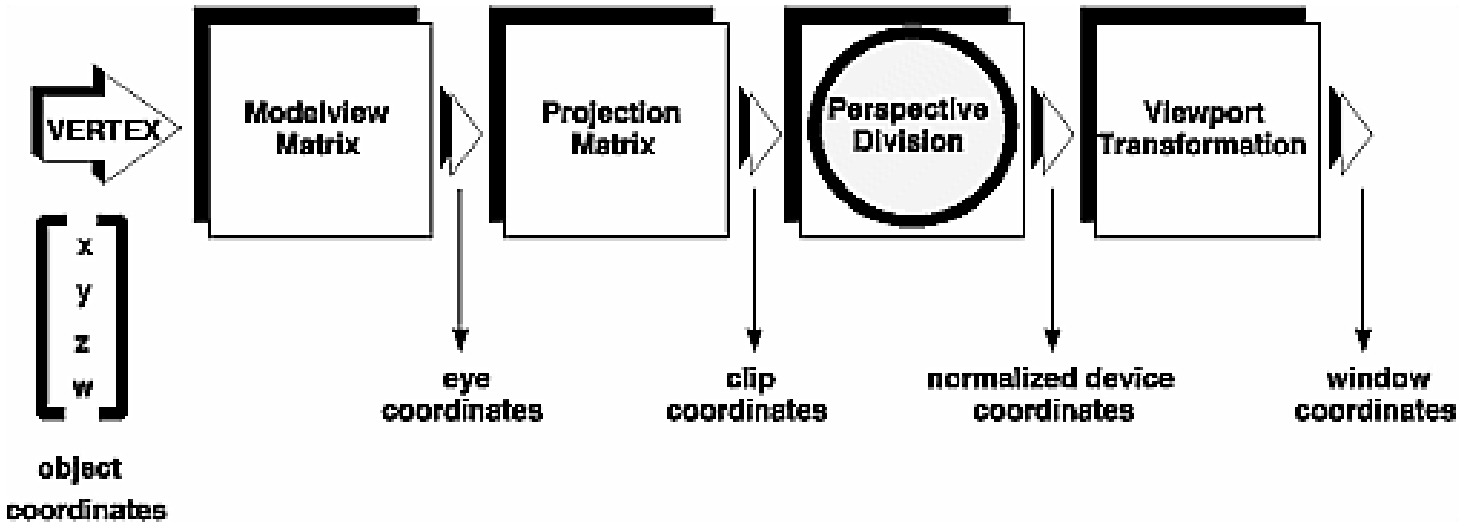

tripod
viewing
positioning the viewing volume in the world
model
modeling
positioning the models in the world
lens
projection
determining shape of viewing volume
photograph
viewport

照相机类比法

- 将照相机固定在三角架上(视图变换)
- 使想要的场景处于取景框中合适的位置(模型变换)
- 调节镜头或调整放大倍数(投影变换)
- 决定最终相片的尺寸大小(视区变换)

照相机类比法

从空间坐标到屏幕坐标的流程



照相机类比法

- 空间坐标经过视图变换和模型变换得到成像坐标
- 经过矩阵变换得剪裁坐标
- 经过透视细分,得到归一化的设备坐标
- 经过视区变换得到窗口坐标

例子一

```
Void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0,1.0,1.0);
    glLoadIdentity();//清空矩阵
    gluLookAt(0.0,0.0,5.0,0.0,0.0,0.0,0.0,0.1,0.0);
    glScalef(1.0,2.0,1.0);
    gluWireCube(1.0);
    glFlush();

};
```

视图变换和模型变换

视图变换和模型变换相当于将空间中一点乘以一个矩阵

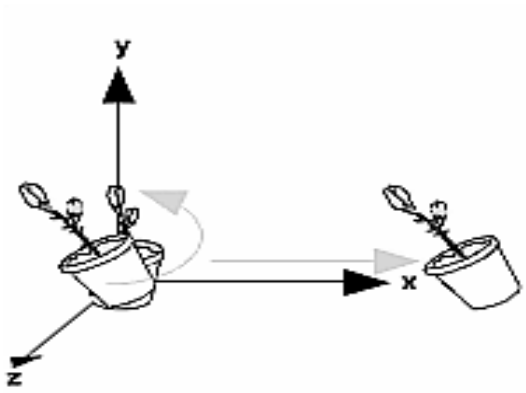
$$\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}$$

object coordinates

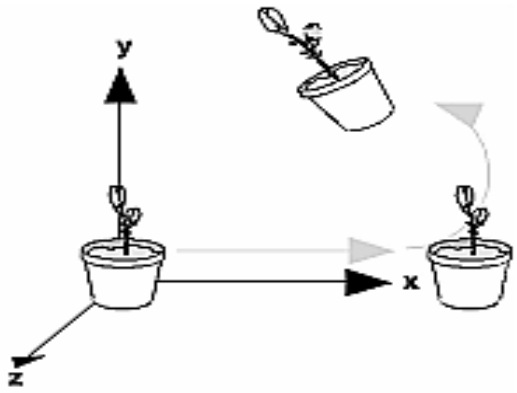
$$M = \begin{bmatrix} m_1 & m_5 & m_9 & m_{13} \\ m_2 & m_6 & m_{10} & m_{14} \\ m_3 & m_7 & m_{11} & m_{15} \\ m_4 & m_8 & m_{12} & m_{16} \end{bmatrix}$$

视图变换和模型变换

变换是有顺序的,矩阵相乘就是有顺序的。



Rotate then Translate



Translate then Rotate

视图变换和模型变换

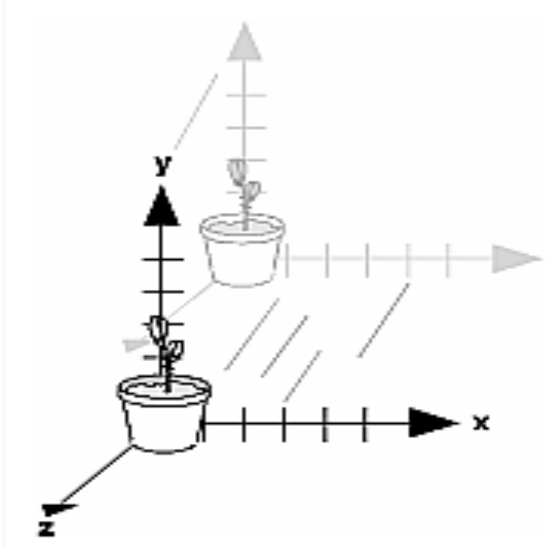
三种模型变换：

- 平移变换
- 旋转变换
- 缩放变换

视图变换和模型变换

平移变换：

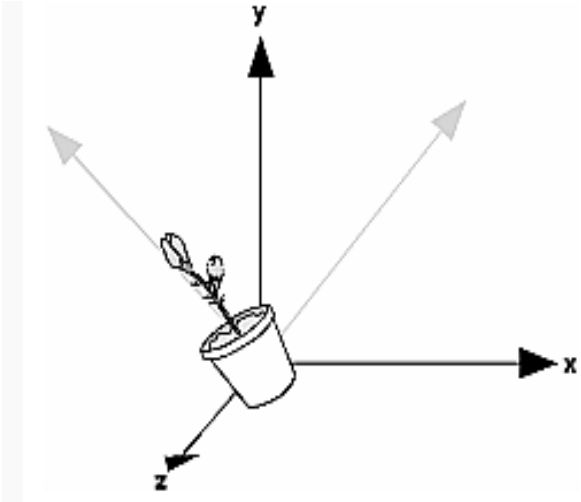
glTranslate{fd}(x,y,z);



视图变换和模型变换

旋转变换：

glRotate{fd}(angle,x,y,z);



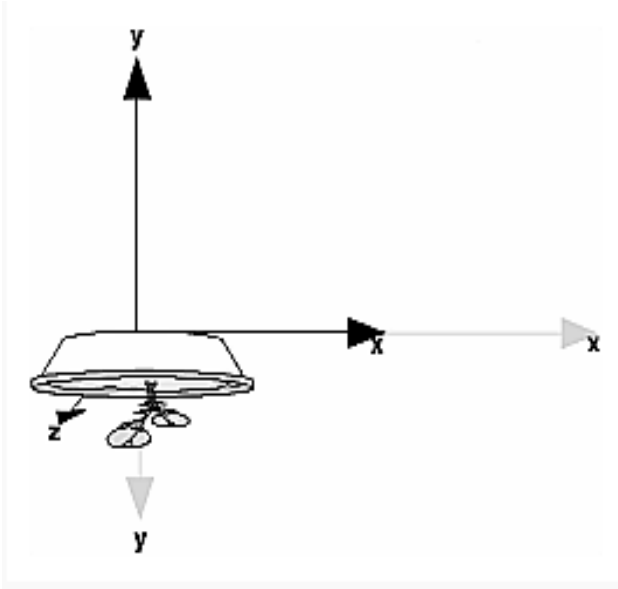
例：

glRotatef(45.0,0.0,0.0,1.0)

视图变换和模型变换

缩放变换：

glScalef(x,y,z)



例：glScalef(2.0,-0.5,1.0)

例子二

显示不同模型变换后的三角形

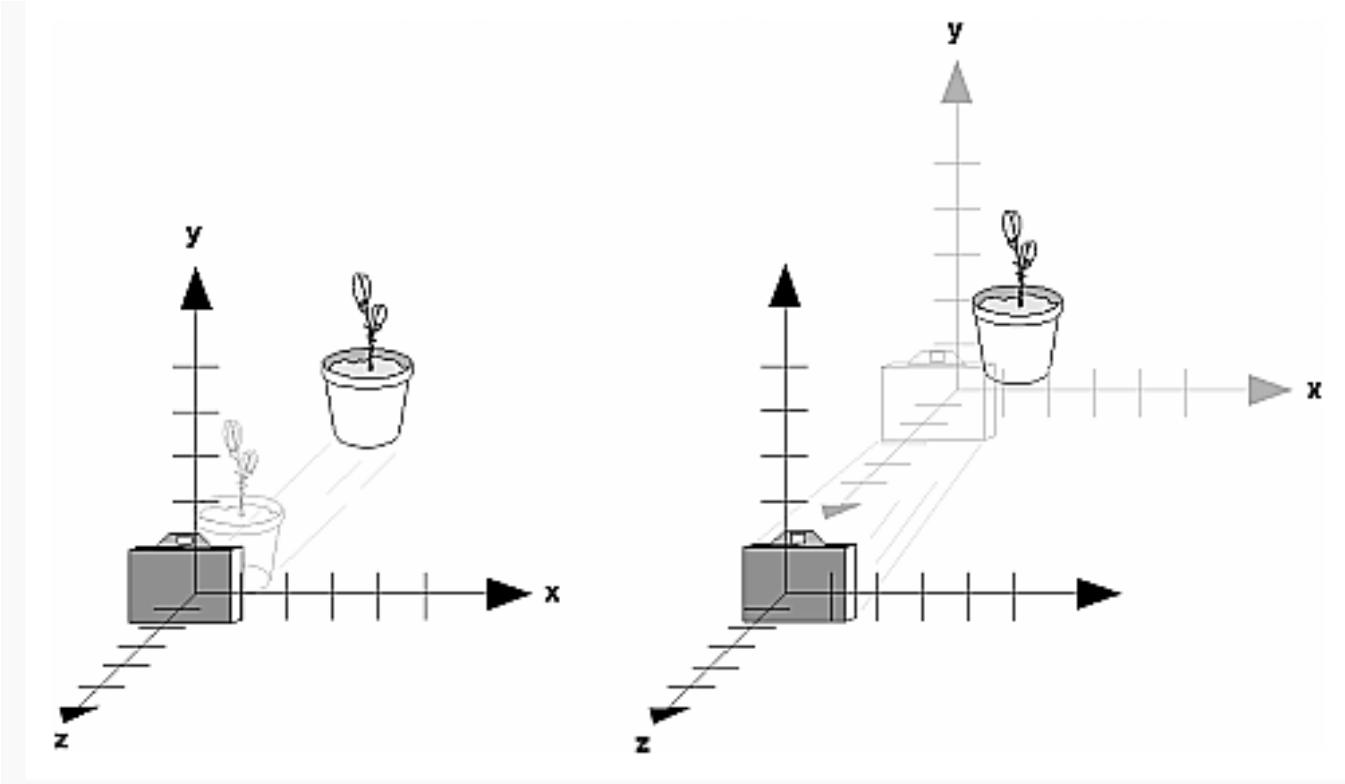
Model.c

视图变换和模型变换

视图变换与模型变换可以互逆变换

- 平移视点相当于逆方向平移模型
- 顺时针旋转视点相当于逆时针旋转物体

视图变换和模型变换

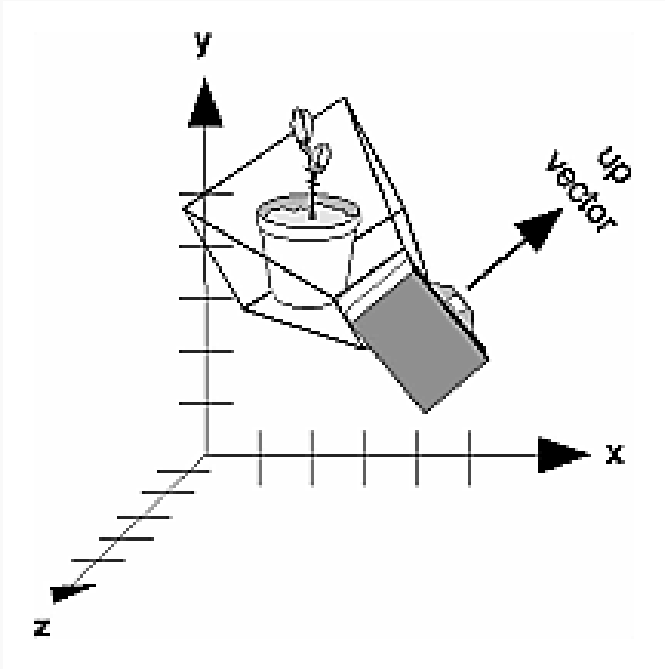


视图变换和模型变换

使用gluLookAt()函数：

void gluLookAt(GLdouble eyex,GLdouble eyey,GLdouble eyez, GLdouble centerx, GLdouble centery, GLdouble centerz, GLdouble upx, GLdouble upy, GLdouble upz)；

视图变换和模型变换




```
gluLookAt(4.0, 2.0, 1.0, 2.0, 4.0, -3.0, 2.0, 2.0, -1.0);
```

视图变换和模型变换

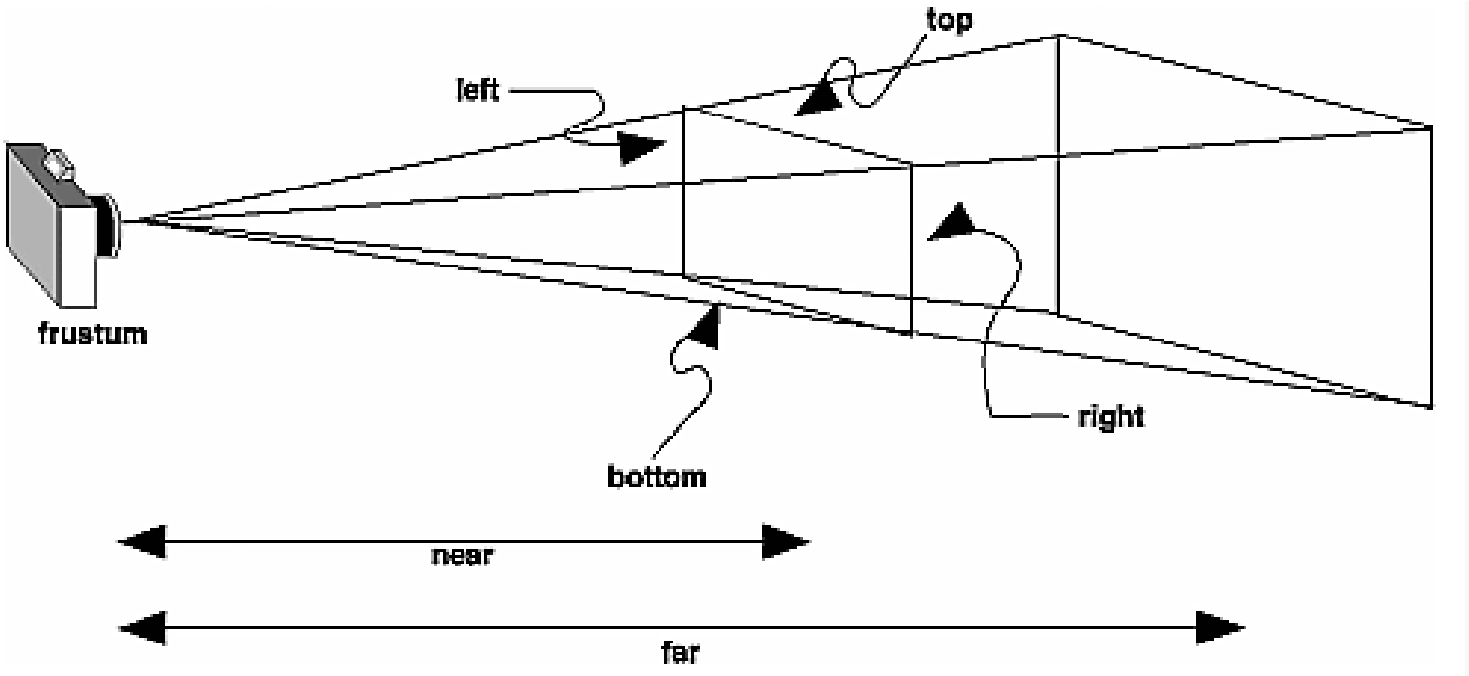
使用以下函数：

```
glMatrixMode(GL_PROJECT);
glLoadIdentity();
glMultMatrixf(M);
```

视图变换和模型变换

透视投影：

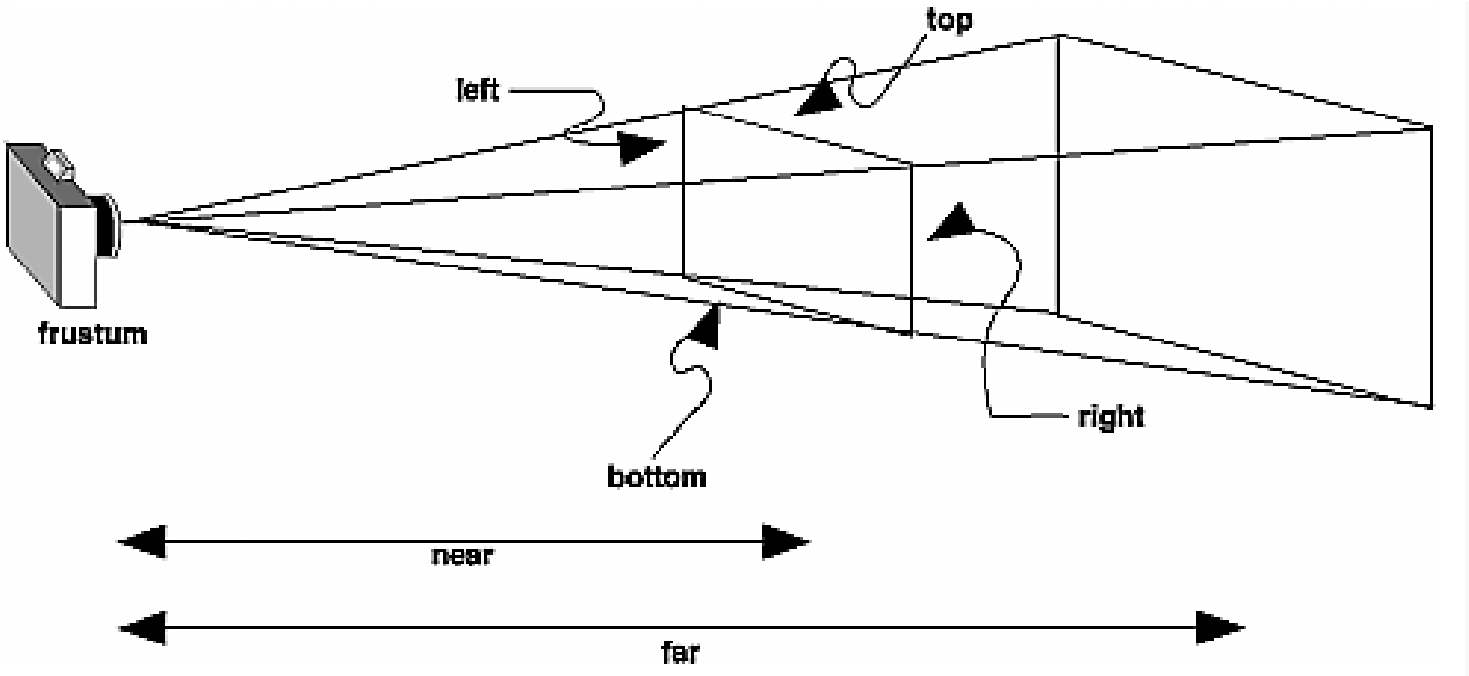
glFrustum(GLdouble left, GLdouble right, GLdouble bottom，GLdouble top，GLdouble near，GLdouble far)；



视图变换和模型变换

透视投影：

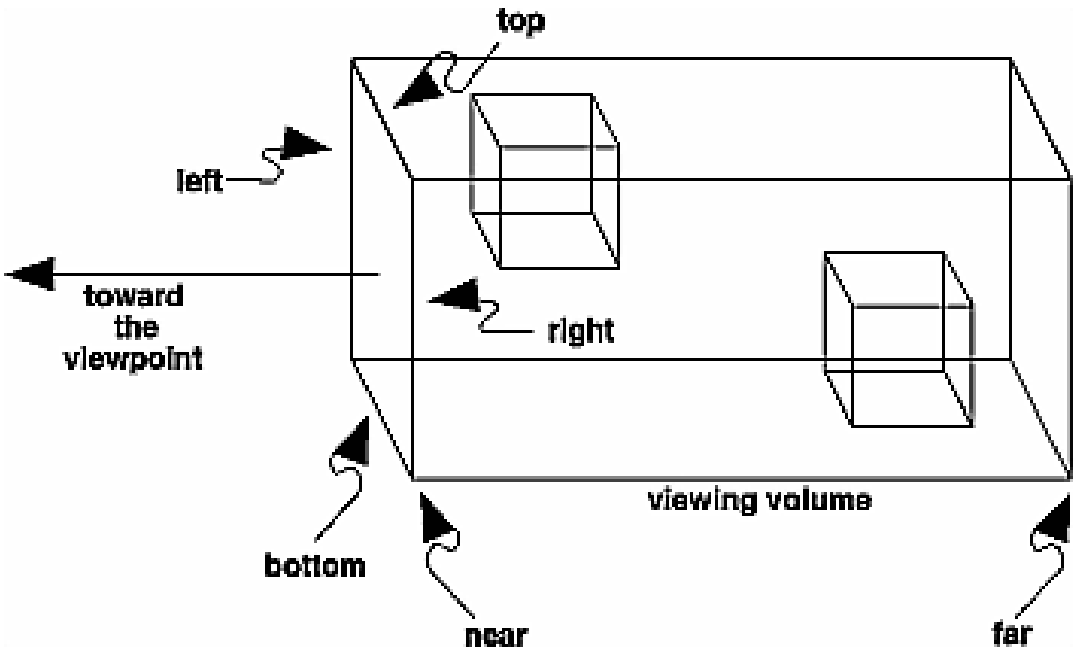
glPerspective(GLdouble fogy, GLdouble aspect, GLdouble near，GLdouble far)；



视图变换和模型变换

正交投影：

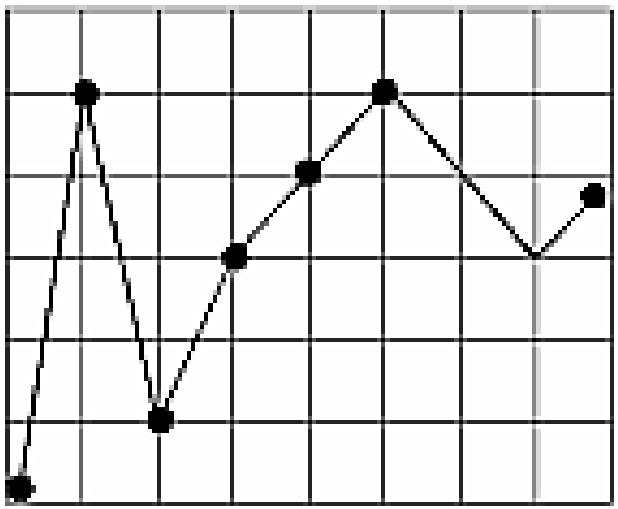
glOrtho(GLdouble left, GLdouble right, GLdouble bottom，GLdouble top， GLdouble near，GLdouble far)；

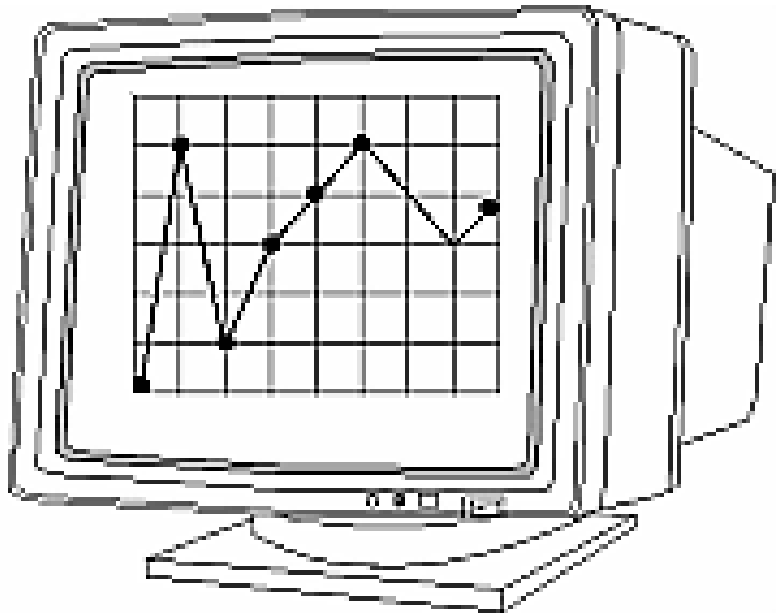


视区变换

视区：

视区是用来绘制图像的窗口的一个矩形区域。

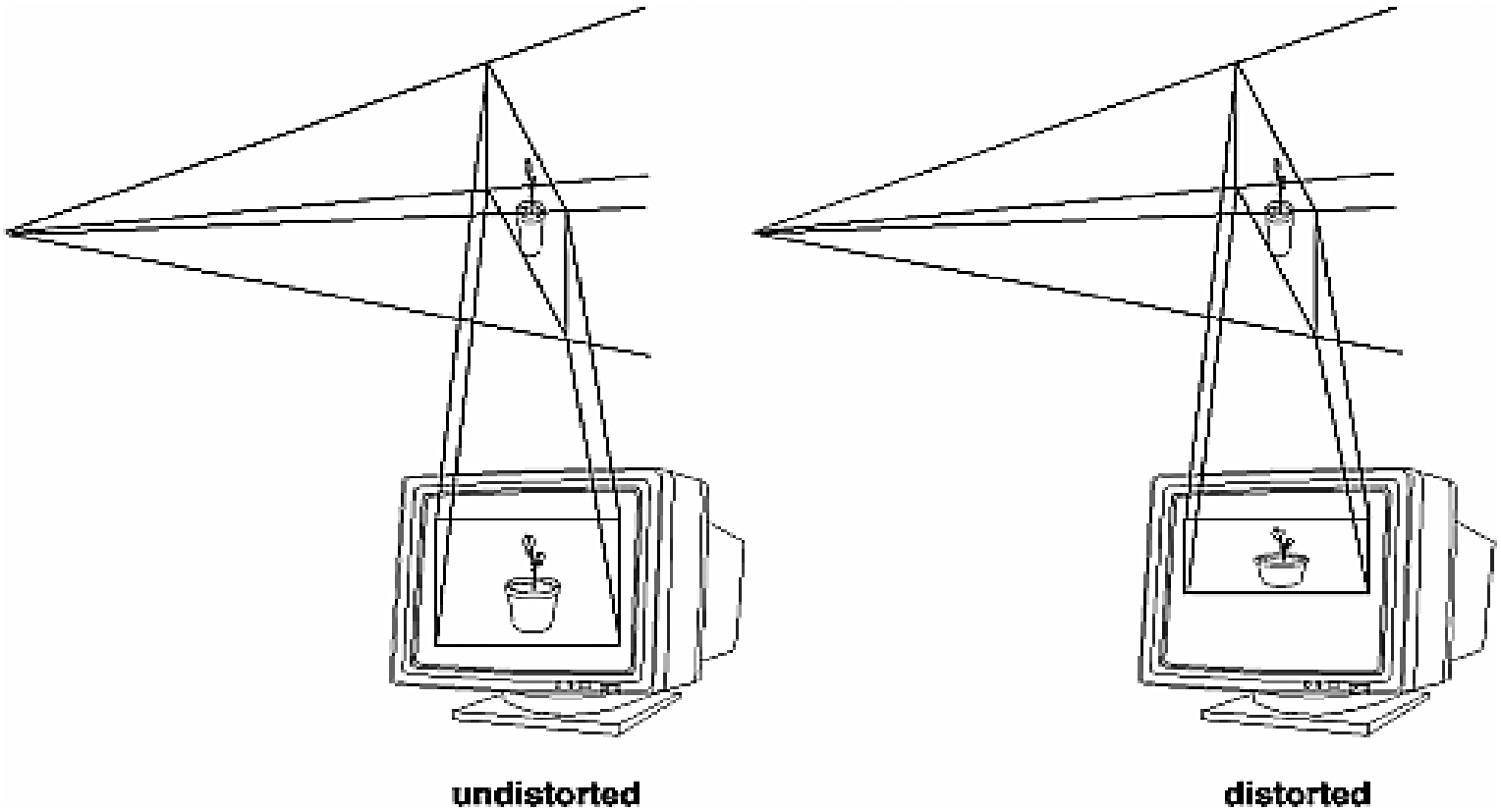




视区变换

定义视区：

void glViewport(GLint x, GLint y, GLint width, GLint height)。



变换的诊断

在实际生活中，让照相机的方向指向正确的位置是很容易的。

编写程序时，往往因为什么错误，导致在计算机屏幕上什么也画不出，从而导致——黑屏效果。

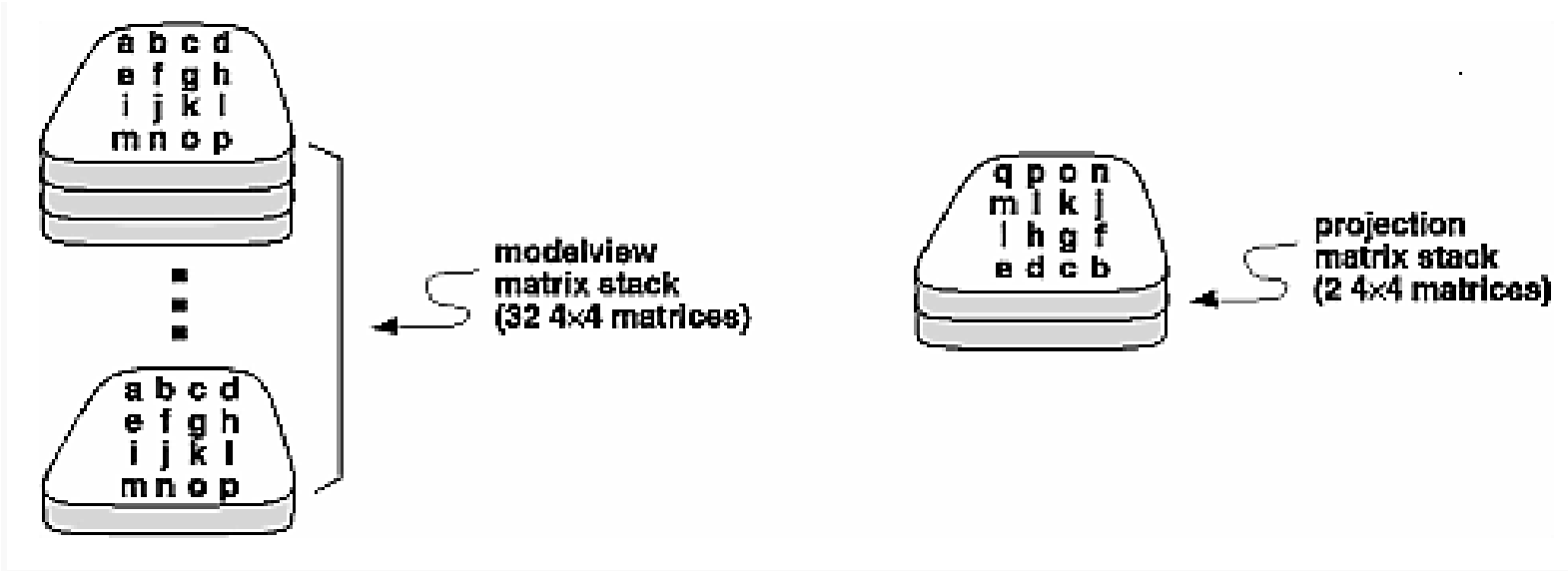
变换的诊断

主要从以下几个方面考虑

- 是否保证使用了不同于屏幕底色的颜色
- 对于投影函数，是否负z轴方向
- 物体是否在剪裁区域内
- 复杂的模型变换时，旋转要慎重
- 透视投影时近剪裁面和远剪裁面之间的位置要设定好

矩阵堆栈的操作

OpenGL提供了最大数目为32个的模型变换矩阵堆栈和最大数目为2个的投影矩阵堆栈。



矩阵堆栈的操作

矩阵堆栈对于构造层次型的模型很有帮助，利用它可以简单的模型构造成复杂的物体。

矩阵堆栈的操作

一个简单应用：

假定绘制一辆汽车，主要工作是绘制车身和车轮。

首先画车身，同时记住画完车身后的当前位置A，然后平移到右前轮的位置画车轮。随后对上一个平移做反变换，即得到当前位置为A，记住该位置然后平移到左前轮，绘制车轮……

矩阵堆栈的操作

对矩阵堆栈操作的两个函数：

void glPushMatrix();

void glPopMatrix();

矩阵堆栈的操作

一段代码例子：绘制车身、车轮以及螺钉。

```
Draw_wheel_and_bolts()
{
    long i;
    draw_wheel();
    for(int  i=0;i<5;i++){
        glPushMatrix();
        glRotate(72.0*i,0.0,0.0,1.0);
        glTranslatef(3.0,0.0,0.0);
        draw_bolt();
        glPopMatrix();
    }
};
```

矩阵堆栈的操作

```
draw_body_and_wheel_and_bolts()
{
    draw_car_body();
    glPushMatrix();
        glTranslate(40,0,30);
        draw_wheel_and_bolts();
    glPopMatrix();
    glPushMatrix();
        glTranslate(40,0,-30);
        draw_wheel_and_bolts();
    glPushMatrix();
    ……
};
```

矩阵堆栈的操作

- 模型变换矩阵堆栈，利用glGetIntergerv(GL_MAX_MODELVIEW_STACK_DEPTH,GLint *params);得到堆栈最大数
- 投影矩阵堆栈

  glGetIntergerv(GL_MAX_MODELVIEW_STACK_DEPTH,GLint *params);堆栈最大数

  将模型变换或者投影矩阵压栈时，都要设置一下矩阵类型，而缺省的设置是模型变换矩阵。所以在对投影矩阵压栈时代码如下：

  glMatrixMode(GL_PROJECTION);

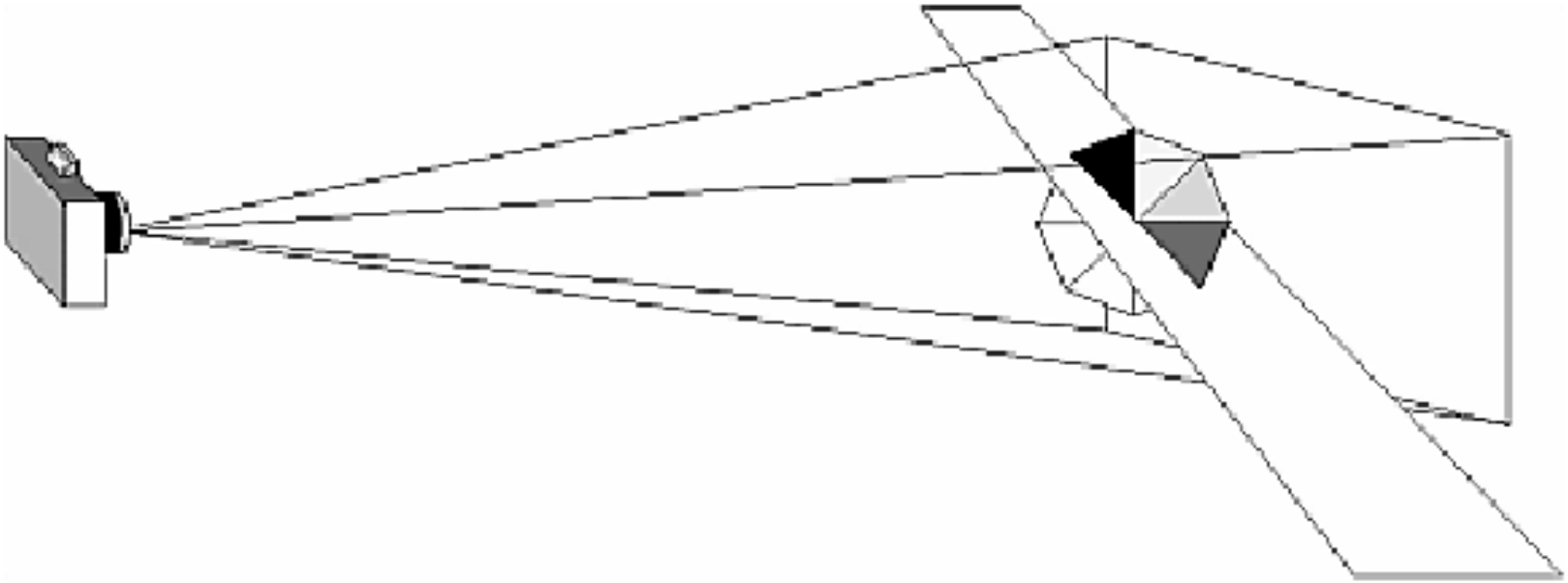
  glPushMatrix();

  …..

  glPopMatrix();

附加的剪裁平面

引入附加剪裁平面的目的是更严格的限制视图体,这种方法对于删除场景中无关的物体非常有用。



附加的剪裁平面

每个平面方程可以表示成Ax+By+Cz+D=0的方式。定义剪裁平面的函数为：

void glClipPlane(GLenum plane，GLdouble *equation)

用以下函数激活定义的剪裁平面：

glEnable(GL_CLIP_PLANEi)

用以下函数取消指定的剪裁平面：

glDisable(GL_CLIP_PLANEi)

附加的剪裁平面

所有满足

$$(A\ \ B\ \ C\ \ D)M^{-1}(x\ \ y\ \ z\ \ w)^T >= 0$$

即表示沿剪裁平面的法向方向上与剪裁体包围的区域是被显示的。

例子三clip.c

利用两个剪裁平面剪裁一个球体

```
/*    clip lower half -- y < 0          */
   glClipPlane (GL_CLIP_PLANE0, eqn);
   glEnable (GL_CLIP_PLANE0);
/*    clip left half -- x < 0           */
   glClipPlane (GL_CLIP_PLANE1, eqn2);
   glEnable (GL_CLIP_PLANE1);
```

组合变换举例

- 建立一个太阳系模型

  绘制一个简单的太阳系，其中有一个太阳，一个行星，均使用相同的球体函数绘制。该程序要模拟行星的公转和自转。

例子四planet.c

- 重点在显示函数

```
glPushMatrix();
glutWireSphere(1.0, 20, 16);
/* draw sun */
glRotatef ((GLfloat) year, 0.0, 1.0, 0.0);   glTranslatef (2.0, 0.0, 0.0);
glRotatef ((GLfloat) day, 0.0, 1.0, 0.0);
glutWireSphere(0.2, 10, 8);
/* draw smaller planet */
glPopMatrix();
```

组合变换举例

- 建立一个有关节的机器人手臂模型

  可以将一个经过缩放以后的长方体做为机器人手臂的一段，但首先必须对每一段作恰当的模型变换。

例子五robot.c

- 显示函数的主要代码

```
glPushMatrix();
glTranslatef (-1.0, 0.0, 0.0);
glRotatef ((GLfloat) shoulder, 0.0, 0.0, 1.0);
glTranslatef (1.0, 0.0, 0.0);
glPushMatrix();
glScalef (2.0, 0.4, 1.0);
glutWireCube (1.0);
glPopMatrix();
```

逆转矩阵

前面讲到的都是将空间中的一点投影到屏幕上一点，即将一个空间三维坐标经过各种变换转换成一个二维的屏幕坐标。而逆转矩阵就是上述过程的逆过程，将一个二维的屏幕坐标转换成空间中的三维坐标。

逆转矩阵

```
int gluUnProject(GLdouble winx, GLdouble
    winy, GLdouble winz,const  GLdouble
    modelMatrix[16],const GLdouble
    projMatrix[16],const GLint
    viewport[4],GLdouble *objx, GLdouble
    *objy, GLdouble *objz);
```

例子unproject.c

练习

- 实现一个由太阳、地球、月亮组成的小系统。